

课程笔记

课程笔记：SysTrace——基于 eBPF 的 AI 全栈性能观测方案

李蔚（华为技术有限公司）

2026 年 4 月 27 日



视频作者/频道：Linux 内核之旅

发布日期：2026-04-26

视频时长：42 分 59 秒

视频链接：<https://www.bilibili.com/video/BV1BYoZBKExk/>

目录

1 算力集群性能诊断的挑战	3
1.1 AI 训练集群的复杂性	3
1.2 训练任务的两大特点	4
1.3 现有监控方案的不足	5
1.4 本章小结	7
2 SysTrace 技术架构与全栈协同分析	8
2.1 从孤立到协同的演进	8
2.1.1 现状：逐层孤立分析的困境	8
2.1.2 目标：全栈协同分析架构	9
2.2 SysTrace 四层架构详解	9
2.2.1 L0: AI 框架层——理解训练在做什么	10
2.2.2 L1: 通信算子层——定位分布式协同瓶颈	11
2.2.3 L2: CANN 层——透视 NPU 内存与算子执行	11
2.2.4 L3: OS 层——捕获系统级干扰	12
2.2.5 四层架构的分工与协同	13
2.3 因果推理链路	13
2.3.1 推理链路的直觉	13
2.3.2 推理链路的正式设计	13
2.3.3 推理链路的实际意义	14
2.4 本章小结	14
3 关键能力：动态分层采集与 eBPF 观测	15
3.1 动态分层采集流程	16
3.1.1 动机：观测深度与性能开销的权衡	16
3.1.2 阶段一：常态化监控（在线正常态）	16
3.1.3 阶段二：慢卡定界（亚健康环境）	16
3.1.4 阶段三：性能定位（管理面根因分析）	16
3.2 eBPF CPU 调度分析	17
3.2.1 动机：谁抢占了 AI 线程的 CPU?	17
3.2.2 机制：通过 <code>sched_switchtracepoint</code>	17
3.2.3 案例：数据加载线程抢占导致 AI 线程饥饿	18
3.3 eBPF 锁争用分析	18
3.3.1 动机：同步瓶颈如何量化?	18
3.3.2 机制： <code>uprobe</code> 动态探测同步原语	18
3.3.3 案例：PyTorch 内存管理中的 GIL 瓶颈	19
3.4 eBPF 内存性能分析	20
3.4.1 动机：内存问题为何难以定位?	20
3.4.2 机制： <code>page_fault</code> 等内核事件的捕获与分析	20
3.4.3 案例：透明大页（THP）导致的性能劣化	21
3.5 本章小结	22

4 效果展示：NPU、内存与 CPU 调度采集	22
4.1 NPU 算子占用故障注入与定位	23
4.1.1 实验动机与配置	23
4.1.2 故障注入方式	23
4.1.3 step 时延变化观测	23
4.1.4 慢卡定界：L0 + L1 数据协同分析	24
4.1.5 案例启示	25
4.2 内存压力故障注入与定位	25
4.2.1 实验动机	25
4.2.2 故障注入方式：NUMA 绑定	26
4.2.3 step 时延变化观测	26
4.2.4 <code>mm_fault</code>	27
4.2.5 根因判定：内存回写错误	28
4.3 CPU 调度干扰故障注入与定位	28
4.3.1 实验动机	28
4.3.2 故障注入方式	29
4.3.3 CPU 调度事件采集与分析	29
4.3.4 根因定位：识别具体进程名	30
4.3.5 案例启示	30
4.4 本章小结	30
5 未来展望：业务拓扑画像与 Agent 调用链观测	31
5.1 LLM 推理部署的观测挑战	31
5.2 基于 eBPF 的动态拓扑构建	32
5.3 AI Agent 调用链观测	34
5.4 本章小结	37
6 总结与延伸	38
6.1 演讲核心要点回顾	38
6.2 Q&A 关键信息提炼	39
6.2.1 使用形式：插件化定位与命令行控制	39
6.2.2 数据分析：从专家经验到大模型推荐	39
6.2.3 按需采集的自动化策略：程序触发与人工控制的结合	40
6.3 技术趋势与开放问题	40
6.3.1 趋势一：从“数据汇总可视化”到“智能根因分析”	41
6.3.2 趋势二：eBPF 在 AI Agent 观测中的前沿价值	41
6.3.3 趋势三：系统层与框架层工具的互补共生	41
6.3.4 开放问题	42
6.4 本章小结	42

1 算力集群性能诊断的挑战



图 1: 演讲标题页：SysTrace 基于 eBPF 的 AI 全栈性能观测方案¹

随着大语言模型（LLM）与生成式 AI 应用的爆发式发展，训练千亿乃至万亿参数规模的模型已成为行业常态。然而，支撑这些模型训练的算力集群在规模与复杂度上急剧膨胀，使得性能诊断与优化成为一项极具挑战的系统工程。本章从集群复杂性、训练任务特征、现有监控短板三个维度，剖析为何必须构建一套全链路、高实时的性能观测体系。

1.1 AI 训练集群的复杂性

全栈结构

现代 AI 训练集群是一个从底层硬件到上层应用、纵向层级极深的**全栈系统**。其典型层次包括：

- **硬件层**：CPU、NPU/GPU 等加速器、高速互连网络（RDMA/NVLink/ 腾 HCCS）；
- **系统软件层**：操作系统内核、驱动、内存管理、调度器；
- **AI 框架层**：PyTorch、MindSpore 等分布式训练框架；
- **应用层**：数据加载（DataLoader）、模型定义、训练脚本。

这一全栈结构意味着：**任一层级的故障或性能劣化，都会向上传导，最终表现为训练时延增加与吞吐量下降**。例如，底层网络的一个微秒级抖动，在 AllReduce 通信算子中被放大后，可能导致整个训练 step 的显著延迟；操作系统调度器的轻微不公平，可能使某张 NPU 卡（慢卡）成为整个数据并行组的瓶颈；数据加载线程的 CPU 抢占，则可能让 GPU 处于空等状态。

¹ 视频画面时间区间：00:00:00-00:00:05。若为裁剪图，时间区间仍对应原视频画面。

从单机到集群的质变

在单机单卡时代，性能瓶颈通常局限于单一维度（如 GPU 利用率不足），开发者只需关注框架层的 Profiling 即可。进入大规模分布式训练后，系统行为呈现高度耦合特征：通信与计算重叠、多卡协同与流水线编排交织、框架调度与内核调度并发。这种复杂性使得“逐层孤立分析”的传统手段难以为继——正如盲人摸象，每个工具只能感知局部，却无法还原全局真相。

因此，理解 AI 训练集群的复杂性，是认识全栈性能观测必要性的第一步。只有将 CPU/NPU、高速网络、框架运行时与应用逻辑视为一个有机整体，才能避免“头痛医头、脚痛医脚”的局部优化陷阱。

1.2 训练任务的两大特点

大规模 AI 训练任务在工程实践中表现出两个极为突出的特点，这两个特点共同决定了性能诊断的紧迫性与经济价值。

特点一：训练周期极长

当前主流的大模型训练周期以**周**甚至**月**为单位计量。以千亿参数规模的模型为例，完成一次完整训练往往需要数周乃至数月的时间。在这一时间尺度下，任何微小的性能抖动都会被累积放大为可观的总开销。

不妨做一个简单的定量分析。设模型总训练步数为 N ，基线单步时延为 T_0 ，若因某性能劣化导致单步时延增加 ΔT ，则总训练时间的增量为：

$$\Delta T_{\text{total}} = N \cdot \Delta T$$

其中：

- N ：训练总步数（由数据集大小与批量大小决定，通常为百万至千万量级）；
- T_0 ：正常情况下的单步时延（**step 时延**）；
- ΔT ：每步额外增加的时延（可能仅数毫秒）。

小抖动，大代价

一个常见的误区是认为“几毫秒的性能损失微不足道”。然而，当 N 达到 10^7 量级时，即使 $\Delta T = 1 \text{ ms}$ ，累积增量也高达 $10^7 \times 10^{-3} = 10^4$ 秒，约合 2.8 小时。在千卡集群上，这相当于浪费数千卡时的算力。若抖动更为严重（如慢卡导致同组其他卡同步等待），代价将呈倍数放大。

特点二：资源成本高昂

大模型训练所消耗的算力资源成本极为惊人。以当前主流云厂商的 NPU/GPU 集群定价估算，一次千亿参数模型的完整训练，算力成本可达数百万至数千万美元。在这一背景下，**资源利用效率直接等同于经济收益**。

然而，实际运行中常常出现两类导致成本激增的问题：

1. **算力资源利用不均**：集群中部分节点（**慢节点**）因硬件故障、散热不良或软件配置差异而性能下降，导致其他正常节点处于空等状态，整体集群利用率被拉低；

2. **性能阻塞与劣化**：通信阻塞、内存交换（swap）、锁争用等问题导致训练任务无法充分利用加速器算力，形成“有卡无用”的局面。

根据业界统计，上述两类问题导致的资源浪费最高可达训练总成本的 **20%–30%**。这意味着，一个本可花费 1000 万元完成的训练任务，可能因未被及时发现的性能问题而额外消耗 200 万至 300 万元。



图 2: 智算集群性能诊断的挑战²

核心认知

训练周期长与成本高这两个特点，共同构成了性能诊断的**经济驱动力**：在长达数周的训练过程中，微小的性能问题不仅会被时间放大，还会直接转化为巨额的资源浪费。因此，**及时发现并定位性能劣化，其本质是一项高回报率的投资。**

1.3 现有监控方案的不足

面对上述挑战，业界并非没有监控手段。然而，当前主流方案存在三个结构性短板，使其难以满足大规模 AI 训练集群的在线性能诊断需求。

²视频画面时间区间：00:02:00–00:02:05。若为裁剪图，时间区间仍对应原视频画面。

短板一：事后复现，缺乏在线常态化监控

许多团队仍采用“训练出问题后再排查”的被动模式。当用户反馈训练速度异常或任务失败时，运维人员需要手动复现问题、收集日志、运行各类诊断工具。这种**事后复现**模式的根本缺陷在于：

- 问题现场往往已被破坏，关键状态信息丢失；
- 复现过程本身消耗额外资源与时间；
- 无法捕捉瞬态或偶发的性能抖动。

理想的监控体系应当是**在线常态化**的——从训练任务启动的那一刻起，观测系统便在后台持续运行，实时采集关键指标，并在性能劣化发生的瞬间触发告警。

短板二：工具割裂，形成数据孤岛

现有的性能分析工具按层级高度分散，彼此之间缺乏数据贯通：

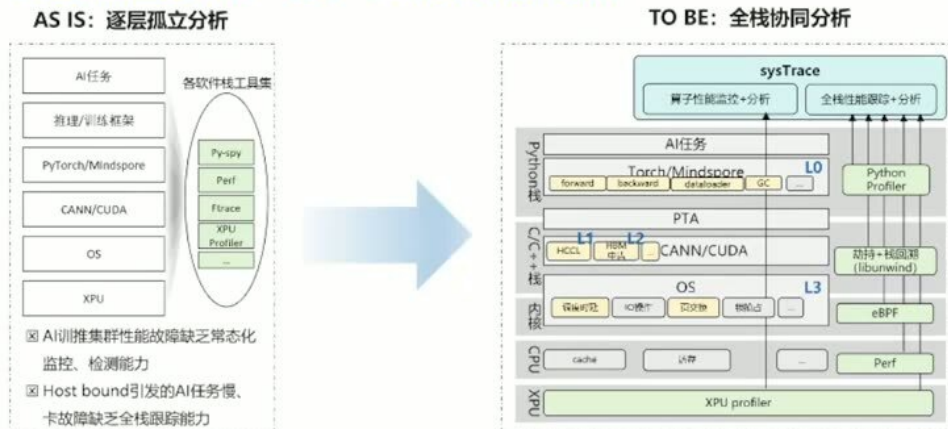
- **应用/框架层**：Python profiler（如 cProfile、PySpy）用于分析 Python 代码热点；
- **系统层**：Perf、Ftrace 用于分析 CPU 性能与内核事件；
- **硬件层**：各类 XPU 厂商提供的 Profiler（如 NVIDIA Nsight、 腾 Profiling）用于分析加速器利用率；
- **网络层**：独立的网络诊断工具分析 RDMA 或 InfiniBand 性能。

每一类工具都生成各自格式的数据，存储于独立的系统中，分析时需要人工关联与比对。这种**数据孤岛**现象带来两个严重后果：首先，跨层级的根因定位极度依赖专家经验，新人难以快速上手；其次，当问题涉及多个层级的交互（如框架通信算子触发内核锁争用）时，孤立工具几乎无法还原完整的因果链条。

第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

从“逐层孤立分析”到“全栈协同分析”



通过构建全栈协同分析体系，打破数据孤岛。将复杂的性能故障定位时间从传统的“数天级”缩短至“小时级”，全面提升系统的稳定性与运维效率。

图 3: 从逐层孤立分析到全栈协同分析的演进³

短板三：定位周期长，无法满足分钟级响应

由于上述两个短板的叠加，当前从发现问题到定位根因的周期通常以天为单位。在训练任务长达数周背景下，数天的定位延迟意味着问题持续造成资源浪费，且可能错过最佳的修复窗口（如检查点保存周期、任务调度间隙等）。

AS IS vs TO BE

现状 (AS IS): 各层级独立分析，工具碎片化，数据分散，依赖人工专家经验串联线索，定位周期数天。

目标 (TO BE): 构建全链路、高实时的性能监测体系，打通从框架层到硬件层的数据通路，实现问题的分钟级发现与精准定位，将诊断周期从“天级”压缩至“小时级”乃至“分钟级”。

1.4 本章小结

本章围绕“为何需要全栈性能观测”这一核心问题，从三个层面展开论述：

1. **AI 训练集群的复杂性：**从 CPU/NPU 到高速网络再到上层框架应用，全栈层级纵深且高度耦合，任一层级的故障均可能向上传导，影响整体训练性能。

³视频画面时间区间：00:03:35-00:03:40。若为裁剪图，时间区间仍对应原视频画面。

2. **训练任务的两大特点**：周期长（数周至数月）与成本高（资源浪费可达 20%-30%），使得微小性能抖动被时间放大为巨额经济损失。
3. **现有监控方案的不足**：事后复现缺乏在线能力、各层级工具割裂形成数据孤岛、定位周期长无法满足实时性要求。

这三方面因素共同指向一个结论：**必须构建一套从全链路到实时在线的性能监测体系，打破数据孤岛，实现性能劣化的分钟级发现与精准定位**。这正是 SysTrace 系统的设计初衷，也是后续章节将详细展开的技术架构与核心能力。

2 SysTrace 技术架构与全栈协同分析

在上一章中，我们已经看到：大规模 AI 训练集群的性能诊断之所以困难，根源在于系统层级复杂、工具割裂、数据孤立。本章将回答一个核心问题——**如何设计一套统一的观测架构，将原本分散在各层的数据打通，实现从上层 AI 任务到底层硬件的全链路协同分析**？我们将从 SysTrace 的整体技术架构出发，理解“从逐层孤立分析到全栈协同分析”的演进思路，以及支撑这一架构的 L0-L3 四层分层观测模型。

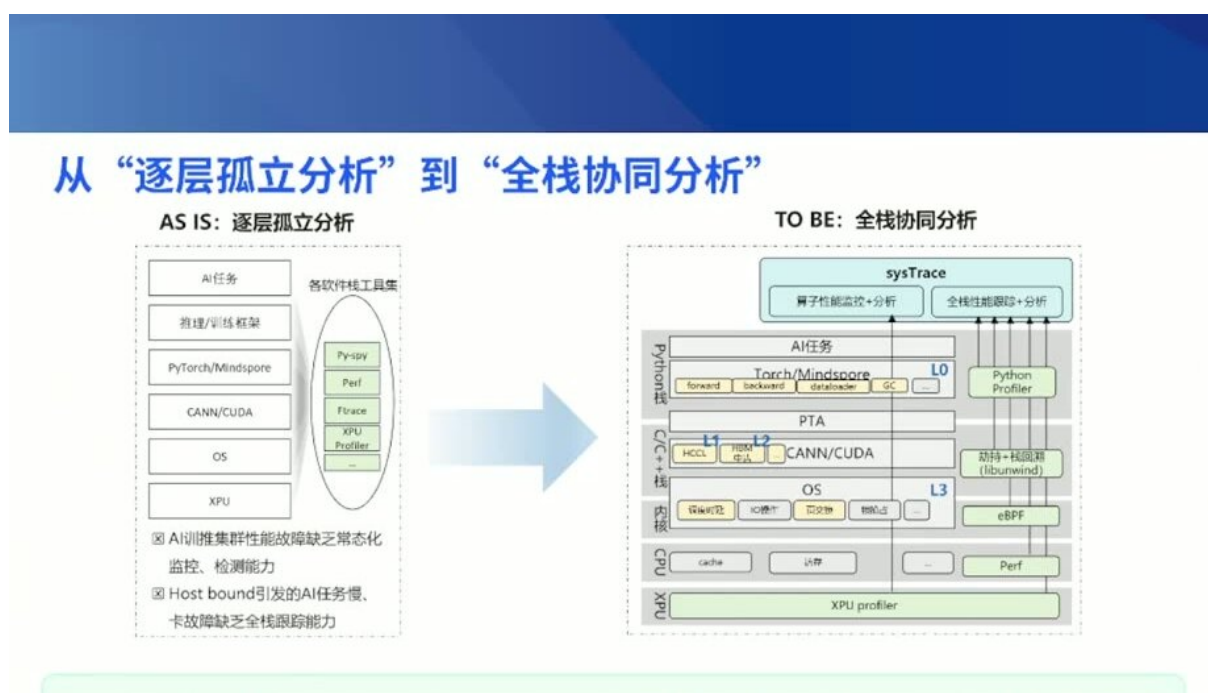


图 4：从逐层孤立分析到全栈协同分析的架构对比⁴

2.1 从孤立到协同的演进

2.1.1 现状：逐层孤立分析的困境

在 GPU 或 NPU 集群上部署大模型训练任务时，性能劣化是常态而非例外。然而，当前业界用于监控这些性能变化的手段，本质上是一种**事后复现**的模式：训练任务出现问题后，工程师需要

⁴ 视频画面时间区间：00:03:55-00:04:00。若为裁剪图，时间区间仍对应原视频画面。

手动登录节点、逐层排查、分别采集数据。更根本的问题在于，从 AI 任务到训练框架，再到底层的 OS 与 XPU，每个层级都使用完全不同的工具链和软件架构进行数据采集。

- **框架层**：使用 Python profiler（如 cProfile、PySpy）分析 Python 代码热点；
- **C/C++ 运行时层**：使用 Perf、Ftrace 等工具追踪原生代码执行；
- **OS 内核层**：使用 Ftrace、SystemTap 或自定义内核模块采集调度、IO 事件；
- **XPU 硬件层**：使用各厂家提供的专用 Profiler（如 NVIDIA Nsight、华为 CANN Profiling）。

这些工具彼此独立、数据格式各异、时间基准难以对齐，形成了典型的**数据孤岛**。当训练任务出现 step 时延抖动时，框架工程师看到的是 Python 层耗时正常，系统工程师看到的是 CPU 调度无异常，而硬件工程师看到的是 XPU 利用率偏低——三个层级的数据无法关联，导致故障定位往往需要**数天甚至更长时间**。

数据孤岛的危害

数据孤岛不仅意味着”工具多”，更意味着**因果关系断裂**。当上层观测到性能劣化时，下层各层级的独立数据无法回答”是哪个层级的哪个事件导致了上层的抖动”。这正是 SysTrace 试图解决的核心问题。

2.1.2 目标：全栈协同分析架构

SysTrace 的设计目标非常明确：构建一个**从上层 AI 任务到底层 XPU 的统一监控架构**，在各层级采集数据的同时，建立从上层行为到下层根因的因果推导链路。这一架构的核心特征包括：

1. **统一时间基准**：所有层级的采集数据基于同一套时间戳体系，确保跨层级事件可关联；
2. **分层按需采集**：不同层级根据诊断需要动态开启或关闭，平衡观测深度与性能开销；
3. **因果推理链路**：不仅采集原始数据，更设计从 L0 到 L3 的推导路径，将”现象”转化为”根因”。

全栈协同分析的核心价值

通过全栈协同分析，SysTrace 将性能故障定位时间从传统的”数天级”缩短至”小时级”。这一数量级的提升，并非来自单一技术的突破，而是来自**架构层面的系统性整合**——将原本割裂的工具链和数据流统一到一个框架之下。

2.2 SysTrace 四层架构详解

为了实现全栈协同分析，SysTrace 将观测对象划分为四个逻辑层级，从最接近 AI 业务的上层框架，逐步下沉到 OS 内核和 XPU 硬件。每一层有明确的采集目标、适用的技术工具，以及与其他层级的协作关系。

全栈协同分析”

TO BE: 全栈协同分析

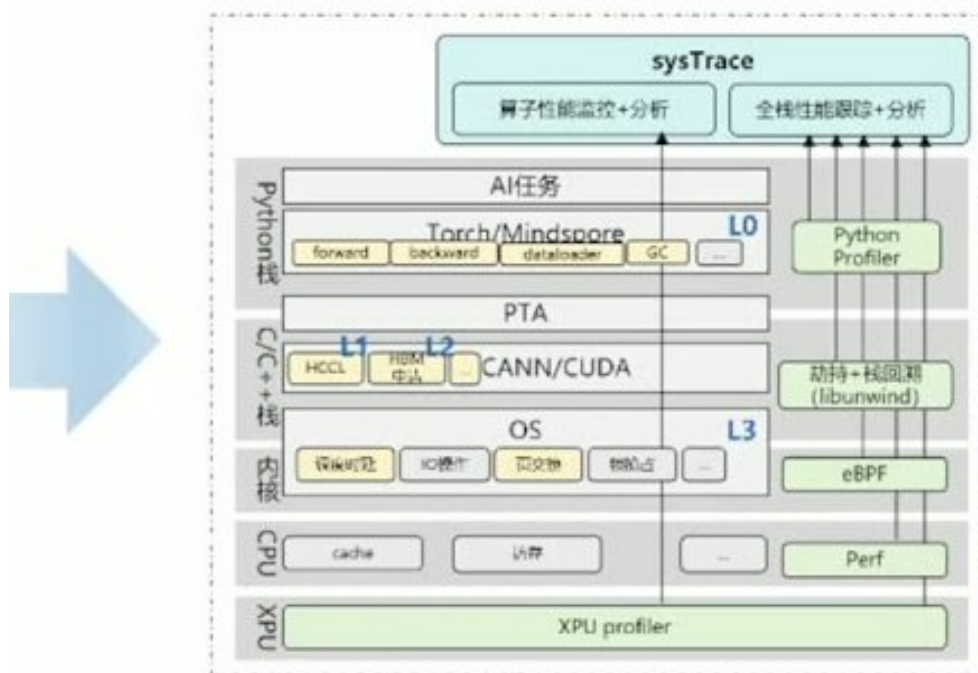


图 5: SysTrace 整体系统架构⁵

2.2.1 L0: AI 框架层——理解训练在做什么

L0 层是最接近 AI 算法工程师的观测层级，关注的是**训练任务在框架层面的关键阶段**。

L0 层采集内容

- **dataloader:** 数据加载与预处理耗时
- **forward:** 前向传播计算耗时
- **backward:** 反向传播与梯度计算耗时
- **optimizer step:** 参数更新耗时
- **GC (垃圾回收):** Python 垃圾回收触发的时间点与耗时

⁵视频画面时间区间：00:03:55-00:04:00。若为裁剪图，时间区间仍对应原视频画面。

L0 层的核心问题是：当 step 时延增加时，是数据加载变慢了，还是计算变长了，抑或是 Python GC 导致了停顿？为此，SysTrace 在框架层使用 Python profiler (PySpy) 进行采集。PySpy 的优势在于对运行中的 Python 进程进行非侵入式采样，无需修改训练代码或重启进程，即可获取 Python 调用栈的时间分布。

```
1 # PySpy以固定频率（如100Hz）采样目标进程的Python调用栈
2 # 输出格式：时间戳 + 调用栈深度 + 函数名
3 1234.56s  main_train_loop
4 1234.56s    Trainer.training_step
5 1234.56s      forward_pass
6 1234.56s        torch.nn.Linear.forward
7 1234.57s  garbage_collect  <-- GC停顿点
```

Listing 1: PySpy 采集的 Python 调用栈示例（示意）

通过 L0 层数据，SysTrace 首先判断性能劣化发生在训练的哪个阶段。如果 forward/backward 耗时正常而 dataloader 占比突增，则问题可能出在数据预处理或 IO 层面；如果 forward 内部某算子耗时异常，则需要结合 L1 或 L2 层进一步定位。

2.2.2 L1：通信算子层——定位分布式协同瓶颈

大规模模型训练普遍采用数据并行（DP）、张量并行（TP）和流水线并行（PP）等分布式策略，这些策略的核心是跨卡、跨节点的集合通信。L1 层专门采集这些通信算子的执行数据。

L1 层采集内容

- AllReduce：梯度同步的总量与耗时
- AllGather：参数聚合的总量与耗时
- ReduceScatter：分散归约的总量与耗时
- Broadcast：参数广播的总量与耗时
- 通信算子的发起时间、完成时间、参与卡号

L1 层的技术实现依赖于**动态库插桩**。SysTrace 在 C/C++ 通信库（如 HCCL、NCCL）的关键函数入口和出口处，通过 libunwind 等工具插入探针，记录通信算子的起止时间和数据量。libunwind 是一个用于获取程序调用栈的开源库，支持在运行时动态解析函数地址与符号名，非常适合对编译后的二进制进行轻量级插桩。

L1 层的核心价值在于**慢卡定界**。当 step 时延增加时，SysTrace 比较各卡上通信算子的完成时间：如果某张卡上的 AllReduce 完成时间显著晚于其他卡，则该卡很可能是“慢卡”（straggler card），需要进一步下沉到 L2 或 L3 层分析其变慢的原因。

2.2.3 L2：CANN 层——透视 NPU 内存与算子执行

L2 层位于 AI 框架与 OS 之间，对应华为昇腾 AI 处理器的 CANN (Compute Architecture for Neural Networks) 软件栈。这一层关注的是 NPU 内部的资源使用，尤其是**高带宽内存(HBM)**的申请、释放与算子执行细节。

L2 层采集内容

- HBM 内存的申请与释放事件（时间点、大小、堆栈）
- 算子在 AI Core 上的执行耗时与调度队列状态
- OOM（Out of Memory）事件的触发点与上下文
- 内存碎片与堆栈分配模式（用于火焰图分析）

L2 层主要使用 **XPU Profiler**（各厂家提供的硬件级性能分析工具）进行采集。以华为昇腾为例，CANN Profiling 可以导出算子级别的执行时间线、内存使用曲线和 AI Core 利用率。SysTrace 将这部分数据与 L0、L1 层的时间轴对齐，从而回答：**慢卡是因为某个算子在 NPU 上执行变慢了，还是因为 HBM 申请阻塞了算子下发？**

L2 层的特殊性

L2 层的数据高度依赖硬件平台（NPU/GPU/TPU），不同厂家的 Profiler 接口和数据格式差异较大。SysTrace 在这一层的设计策略是**抽象共性、适配差异**：统一采集 HBM 事件和算子耗时等通用指标，具体实现则通过各厂家的 Profiler SDK 完成。

2.2.4 L3：OS 层——捕获系统级干扰

L3 层是最底层的观测层级，关注**操作系统内核中的关键事件**。许多 AI 训练性能问题的根因并不在 AI 代码本身，而在 OS 层面的资源竞争——例如，非 AI 进程抢占了 CPU、内存交换导致缺页频繁、锁争用导致线程阻塞等。

L3 层采集内容

- **CPU 调度**：线程切换（sched_switch）、运行队列长度、调度延迟
- **IO 事件**：块设备读写、文件系统操作、页缓存命中率
- **内存管理**：缺页中断（page_fault）、内存交换（swap）、透明大页（THP）活动
- **锁与同步**：内核锁争用、用户态锁（如 Python GIL）的等待时间

L3 层是 **eBPF 技术** 的核心用武之地。eBPF 程序以内核模块的形式运行，可以挂载到 tracepoint、uprobe、kprobe 等探测点上，以低于 1% 的开销实时采集上述事件。例如：

- 通过 sched_switch tracepoint 追踪 AI 线程何时被切换出 CPU、被哪个进程抢占；
- 通过 page_fault tracepoint 统计各进程的缺页频率，识别内存压力节点；
- 通过 uprobe 动态探测 PyTorch 框架中的锁操作，量化 GIL 或其他自定义锁的等待时间。

```
1 // 挂载到 sched_switch tracepoint
2 SEC("tp/sched/sched_switch")
3 int trace_sched_switch(struct trace_event_raw_sched_switch *ctx) {
4     u32 prev_pid = ctx->prev_pid;
5     u32 next_pid = ctx->next_pid;
6     u64 ts = bpf_ktime_get_ns();
```



```

7
8 // 记录AI线程被切换出CPU的事件
9 if (is_ai_thread(prev_pid)) {
10     record_sched_out(prev_pid, ts, next_pid);
11 }
12 return 0;
13 }

```

Listing 2: eBPF 程序示例：追踪 sched_switch 事件（示意）

L3 层的数据为全栈分析提供了**最后的拼图**：当 L0 层发现 step 时延增加、L1 层定位到某张慢卡、L2 层排除算子执行问题后，L3 层可以揭示”慢卡”背后的系统级根因——例如，某非 AI 进程占用了该卡所在节点的 CPU，导致数据预处理线程饥饿，进而拖慢了通信算子的数据准备。

2.2.5 四层架构的分工与协同

将 L0–L3 四层放在一起，可以清晰地看到 SysTrace 的分层设计逻辑：

表 1: SysTrace L0–L3 四层架构对比

层级	观测对象	核心技术	关键问题	输出
L0	AI 框架关键阶段	PySpy (Python profiler)	训练在哪个阶段变慢？	阶段耗时分布
L1	通信算子数据	libunwind 插桩	哪张卡是慢卡？	通信时延对比
L2	CANN 层/HBM	XPU Profiler	NPU 内部出了什么问题？	内存/算子火焰图
L3	OS 内核事件	eBPF	系统级干扰是什么？	调度/IO/锁事件

四层之间不是简单的”堆叠”，而是**有机的协同**：L0 发现问题现象，L1 缩小范围到具体卡号，L2 和 L3 分别从上（NPU）和下（OS）两个方向逼近根因。这种设计使得 SysTrace 在面对复杂性能问题时，能够以最小的开销、最高的效率完成定位。

2.3 因果推理链路

分层采集解决了”数据从哪来”的问题，但全栈协同分析的核心挑战在于**如何将这些数据关联起来，形成从现象到根因的推导路径**。SysTrace 为此设计了一条清晰的因果推理链路。

2.3.1 推理链路的直觉

想象一个医生诊断病人的过程：病人说”我头疼”（症状，对应 L0 的 step 时延增加），医生先检查血压（L1，排除血管问题），再检查颅内是否有异常（L2，排除器质性病变），最后询问最近是否睡眠不足（L3，发现生活作息干扰）。SysTrace 的因果推理链路遵循类似的逻辑——**从宏观症状出发，逐层排除，最终锁定根因**。

2.3.2 推理链路的正式设计

SysTrace 的因果推理链路可以形式化地描述为以下步骤：

1. 症状检测（L0 层）：

- 指标：step 时延 T_{step} 的基线值 T_{base} 与当前值 T_{curr}

- 判据：若 $T_{\text{curr}} > T_{\text{base}} \times (1 + \theta)$ (θ 为预设阈值，如 10%)，则触发分层采集
- 输出：性能劣化告警，附带 L0 各阶段（dataloader/forward/backward/GC）耗时分布

2. 范围定界 (L1 层):

- 指标：各卡通信算子完成时间 t_i^{comm} , $i \in \{0, 1, \dots, N-1\}$
- 判据：计算各卡通信时延与均值的偏差 $\delta_i = t_i^{\text{comm}} - \bar{t}^{\text{comm}}$ ，若某 $\delta_j \gg \delta_{i \neq j}$ ，则卡 j 为慢卡
- 输出：慢卡编号 j 及其通信时延异常倍数

3. 根因定位 (L2 + L3 层):

- L2 方向：检查慢卡 j 的 HBM 申请释放模式、算子执行时间线，判断是否为 NPU 内部问题
- L3 方向：检查慢卡 j 所在节点的 CPU 调度事件、内存缺页率、锁等待时间，判断是否为 OS 层干扰
- 综合：若 L2 发现某算子执行时间突增，则根因为 NPU 侧；若 L3 发现非 AI 线程抢占 CPU 或内存交换活跃，则根因为 OS 侧

因果推理的核心思想

因果推理链路的设计遵循“先定界、后定位”的原则。L0 回答“有没有问题”，L1 回答“问题在哪张卡”，L2 和 L3 回答“问题的原因是什么”。每一层的判据都基于可量化的指标和阈值，避免主观判断，确保推理过程可复现、可自动化。

2.3.3 推理链路的实际意义

这条因果推理链路的价值不仅在于理论上的优雅，更在于工程上的实用性。在没有 SysTrace 之前，工程师需要手动登录节点、分别运行 Python profiler、通信库日志分析工具、OS 性能工具，然后凭经验将多份报告“拼”在一起。SysTrace 通过统一的架构设计，将这一过程**自动化、系统化**：

- 数据采集自动化：各层探针由 SysTrace 统一管理和触发；
- 时间对齐自动化：所有事件基于同一套单调时钟，跨层级关联无需人工对齐；
- 推理路径结构化：从 L0 到 L3 的推导顺序固定，避免了“眉毛胡子一把抓”的盲目排查。

2.4 本章小结

本章系统介绍了 SysTrace 的技术架构与全栈协同分析的设计思想，核心要点如下：

1. **从孤立到协同的演进**：当前 AI 训练性能诊断面临数据孤岛困境，各层级工具割裂导致故障定位耗时数天。SysTrace 通过统一架构将框架层、通信层、CANN 层和 OS 层的数据整合，实现全栈协同分析。

2. L0-L3 四层架构:

- L0（框架层）：使用 PySpy 采集 dataloader/forward/backward/GC 等关键阶段；

- L1 (通信层): 使用 libunwind 插桩采集 AllReduce/AllGather 等通信算子, 实现慢卡定界;
- L2 (CANN 层): 使用 XPU Profiler 采集 HBM 申请释放与算子执行, 透视 NPU 内部;
- L3 (OS 层): 使用 eBPF 采集调度/IO/内存/锁事件, 捕获系统级干扰。

3. **因果推理链路**: SysTrace 设计了从 L0 症状检测 → L1 范围定界 → L2+L3 根因定位的结构化推导路径, 将性能故障定位从”经验驱动”转变为”数据驱动”。

4. **核心价值**: 全栈协同分析将故障定位时间从”数天级”缩短至”小时级”, 其关键不在于单一技术的先进性, 而在于**架构层面的系统性整合与跨层级因果推理能力**的设计。

在下一章中, 我们将深入 SysTrace 的两大关键能力——常态化监控与动态分层采集, 以及 eBPF 在 CPU 调度、锁争用、内存性能三个维度的具体观测机制。

3 关键能力：动态分层采集与 eBPF 观测

在前一节中, 我们了解了 SysTrace 的整体架构与 L0-L3 四层观测模型。然而, 仅有分层架构还不足以支撑大规模训练集群的常态化运维——如果每一层都全天候全量采集, 带来的性能开销将不可忽视; 如果仅在事后手动触发, 又无法捕捉稍纵即逝的抖动。因此, SysTrace 设计了两大关键能力: **常态化监控与动态分层采集**。前者负责在后台以极低开销持续“巡检”, 后者则在检测到异常时按需“深挖”, 逐层开启细粒度采集。与此同时, eBPF 作为 L3 层的核心技术, 在 CPU 调度、锁争用、内存性能三个维度提供了内核级的精准观测手段。本节将深入剖析这套机制的设计动机、实现原理与实际案例。



图 6: SysTrace 动态分层采集流程⁶

⁶视频画面时间区间: 00:05:45-00:05:50。若为裁剪图, 时间区间仍对应原视频画面。

3.1 动态分层采集流程

3.1.1 动机：观测深度与性能开销的权衡

大规模 AI 训练任务通常持续数周甚至数月。若在整个训练周期内对所有层级（L0-L3）进行高频全量采集，不仅会产生海量数据，还会因探针本身的开销干扰训练性能，形成“观测者效应”。反之，若完全依赖事后复现，许多偶发的性能抖动将难以捕捉。因此，SysTrace 的核心设计哲学是：**先轻量巡检，后按需深挖**。

动

态分层采集的本质是一种**自适应观测策略**：以常态化监控为“触发器”，以分层采集为“放大器”，在开销与精度之间动态平衡。

3.1.2 阶段一：常态化监控（在线正常态）

当 AI 训练任务启动后，SysTrace 以插件形式随任务一同运行，在后台通过框架侧接口实时采集 L0 层的关键阶段信息。具体而言，SysTrace 约每 30 分钟（该周期可配置）对框架内的关键阶段进行一次检测，核心指标是 **step 时延**——即一个训练迭代步（forward + backward + 优化器更新）的耗时。

检测算法采用对比策略：将当前时间窗口内的 step 时延与历史基线进行比较，判断是否存在显著劣化。若未发现异常，SysTrace 保持最低限度的采集，对训练任务几乎无感知；若检测到劣化，则进入下一阶段。

3.1.3 阶段二：慢卡定界（亚健康环境）

一旦确认性能劣化，SysTrace 首先尝试**定界**——即定位到具体的故障卡（慢卡）。此时，系统会动态开启 L1 层（通信算子层）的采集，获取 AllReduce、AllGather、ReduceScatter 等集合通信算子的耗时数据。结合 L0 层已有的框架阶段数据，SysTrace 可以判断通信慢是否为导致 step 时延增加的主因，并进一步通过组内特征量相关性比对（如 PP 分组内的卡间时延对比），识别出少数异常节点。

L0-L3 各层在动态采集中的角色

- **L0（框架层）**：常态化监控阶段即开启，提供 step 时延、dataloader、forward、backward 等高层指标。
- **L1（通信层）**：劣化触发后开启，用于慢卡定界，采集通信算子耗时。
- **L2（CANN 层）**：定界后进一步开启，监测 HBM 申请释放、内存堆栈，诊断 OOM 等问题。
- **L3（OS 层）**：最终阶段开启，通过 eBPF 采集调度、IO、页表、锁等内核事件，定位非 AI 线程干扰。

3.1.4 阶段三：性能定位（管理面根因分析）

在锁定慢卡之后，SysTrace 继续向下深挖，逐层开启 L2 与 L3 的采集：

1. **L2 层 (CANN 层)**: 采集 HBM 的申请与释放事件, 分析是否存在内存泄漏或 OOM 风险。若某张卡的 HBM 占用曲线异常攀升或出现频繁分配失败, 则提示内存子系统存在问题。
2. **L3 层 (OS 层)**: 通过 eBPF 采集内核关键事件, 包括 IO 请求、页表操作、CPU 调度切换、锁争用等。这一层的核心目标是识别**非 AI 类线程** (如数据加载进程、系统守护进程、甚至其他用户进程) 对 AI 训练进程的干扰。

最终, SysTrace 将 L0-L3 的数据按时间轴对齐, 在可视化界面上呈现出自上而下的全栈视图。运维人员可以在某一时间窗口内, 逐层查看“每一层都在干什么”, 从而快速定位根因。

动

态分层采集并非简单的“层级开关”, 而是需要**时间轴对齐**与**因果关联**。若各层数据的时间戳未精确同步, 将导致根因定位偏差。SysTrace 通过内核事件的时间戳与上层应用观测对接, 确保跨层数据的一致性。

3.2 eBPF CPU 调度分析

3.2.1 动机: 谁抢占了 AI 线程的 CPU ?

在 AI 训练任务中, 计算密集型线程 (如 NPU/GPU 算子下发线程) 对 CPU 的实时性要求极高。然而, 实际集群环境中常存在非核心线程——例如数据加载 (dataloader) 进程、日志轮转进程、甚至其他用户的计算任务——与 AI 进程竞争 CPU 资源。传统的 top 或 perf 工具只能给出粗粒度的 CPU 占用率, 难以精确刻画“谁在什么时刻抢占了 AI 线程”以及“抢占导致了多长的调度延迟”。

3.2.2 机制: 通过 `schedswitchtracepoint`

eBPF 通过挂载 Linux 内核的 `sched_switch` tracepoint, 可以捕获每一次线程调度切换事件。该 tracepoint 在调度器选择下一个运行线程时触发, 携带以下关键信息:

- 被切换出 CPU 的线程 (`prev_pid`、`prev_comm`、`prev_state`)
- 被切换入 CPU 的线程 (`next_pid`、`next_comm`)
- 事件发生的精确时间戳与 CPU 编号

SysTrace 中的 eBPF 程序挂载于 `sched_switch`, 在每次调度切换时记录上述信息, 并通过 eBPF map 维护每个 AI 线程的运行队列状态。通过分析这些数据, 可以计算出:

- **CPU 占用率**: 某线程在观测窗口内的实际运行时间占比。
- **调度延迟**: 线程从“就绪”到“真正运行”的等待时间。
- **运行队列长度**: 反映 CPU 负载饱和程度。

3.2.3 案例：数据加载线程抢占导致 AI 线程饥饿

在实际场景中，SysTrace 观测到某训练任务的 step 时延周期性抖动。通过 eBPF CPU 调度分析发现，在抖动时刻，名为 `dataloader` 的进程频繁抢占 AI 主线程所在的 CPU 核心，导致 AI 线程的运行队列长度骤增，调度延迟从正常的数毫秒上升至数十毫秒。进一步定位发现，该 `dataloader` 进程因未正确绑定 CPU 亲和性，与 AI 线程发生了资源竞争。

```
1 SEC("tp/sched/sched_switch")
2 int trace_sched_switch(struct trace_event_raw_sched_switch *ctx)
3 {
4     u64 ts = bpf_ktime_get_ns();
5     u32 prev_pid = ctx->prev_pid;
6     u32 next_pid = ctx->next_pid;
7
8     // 记录prev线程本次运行时长
9     struct task_info *prev = bpf_map_lookup_elem(&tasks, &prev_pid);
10    if (prev) {
11        prev->runtime += ts - prev->last_run_ns;
12        prev->state = ctx->prev_state;
13    }
14
15    // 记录next线程开始运行时间，计算调度延迟
16    struct task_info *next = bpf_map_lookup_elem(&tasks, &next_pid);
17    if (next) {
18        next->wait_delay = ts - next->ready_ns;
19        next->last_run_ns = ts;
20    }
21    return 0;
22 }
```

Listing 3: eBPF 程序片段：sched_switch 事件处理与调度延迟计算

`sched_switch` 是理解 CPU 调度行为的“黄金 tracepoint”。它不仅揭示了“谁在运行”，更通过时间戳差值揭示了“等了多久才运行”，是诊断调度类性能问题的关键数据源。

3.3 eBPF 锁争用分析

3.3.1 动机：同步瓶颈如何量化？

Python 作为 AI 框架（如 PyTorch）的主要前端语言，其全局解释器锁（GIL）以及框架内部的各种自定义锁（如内存管理锁、张量操作锁）在高并发场景下极易成为瓶颈。然而，锁争用问题具有高度动态性：争用程度随负载变化，且不同锁的影响范围各异。传统静态分析难以捕捉运行时的锁行为，而 eBPF 的 `uprobe` 机制为此提供了动态探测能力。

3.3.2 机制：uprobe 动态探测同步原语

`uprobe` 是 eBPF 在用户空间函数入口或出口设置的动态探针，无需修改被测程序源码或重新编译。SysTrace 利用 `uprobe` 对 PyTorch 等框架中的锁操作函数进行探测，典型目标包括：

- GIL 获取与释放函数（如 `PyGILState_Ensure` / `PyGILState_Release`）
- 自定义互斥锁的加锁/解锁函数（如 `pthread_mutex_lock` / `pthread_mutex_unlock`）
- 内存分配器的锁操作（如 `malloc` 内部的 arena 锁）

通过在锁函数入口记录时间戳与线程 ID，在出口计算持锁时长，SysTrace 可以量化以下指标：

- **锁等待时间**：线程尝试获取锁到成功获取之间的等待时长。
- **持锁时间**：线程持有锁的时长，过长的持锁时间意味着锁粒度可能过粗。
- **阻塞线程数**：在某一时刻等待同一把锁的线程数量，反映锁的争用激烈程度。

3.3.3 案例：PyTorch 内存管理中的 GIL 瓶颈

在某训练任务的性能分析中，SysTrace 通过 uprobe 探测到 PyTorch 的内存管理路径存在异常的 GIL 争用。具体表现为：多个 dataloader 工作进程在将数据转换为张量时频繁争夺 GIL，导致 AI 主线程长时间无法获取解释器锁，forward 阶段出现周期性卡顿。通过优化数据预处理逻辑（将部分计算移至子进程并减少 GIL 交互），step 时延下降了约 15%。

```

1 SEC("uprobe/pthread_mutex_lock")
2 int trace_mutex_lock(struct pt_regs *ctx)
3 {
4     u32 tid = bpf_get_current_pid_tgid();
5     u64 ts = bpf_ktime_get_ns();
6
7     struct lock_event ev = {};
8     ev.tid = tid;
9     ev.lock_addr = PT_REGS_PARM1(ctx);
10    ev.wait_start_ns = ts;
11    bpf_map_update_elem(&lock_waits, &tid, &ev, BPF_ANY);
12    return 0;
13 }
14
15 SEC("uretprobe/pthread_mutex_lock")
16 int trace_mutex_lock_ret(struct pt_regs *ctx)
17 {
18     u32 tid = bpf_get_current_pid_tgid();
19     struct lock_event *ev = bpf_map_lookup_elem(&lock_waits, &tid);
20     if (ev) {
21         ev->wait_end_ns = bpf_ktime_get_ns();
22         ev->wait_duration_ns = ev->wait_end_ns - ev->wait_start_ns;
23         bpf_perf_event_output(ctx, &lock_events, BPF_F_CURRENT_CPU, ev, sizeof(*ev));
24         bpf_map_delete_elem(&lock_waits, &tid);
25     }
26     return 0;
27 }

```

Listing 4: eBPF 程序片段：uprobe 探测锁函数并计算等待时间

probe 的开销虽低，但在极高频函数（如每秒数百万次的内存分配）上大规模部署仍可能引入性能抖动。SysTrace 采用“按需开启”策略：仅在动态分层采集触发 L3 层后才激活锁争用探针，避免常态化监控阶段的额外开销。



图 7: eBPF 在 AI 性能观测中的三大应用模块⁷

3.4 eBPF 内存性能分析

3.4.1 动机：内存问题为何难以定位？

AI 训练任务对内存（尤其是 HBM 与主机内存）的消耗巨大。内存问题通常表现为三类症状：**OOM (Out of Memory)** 导致训练中断、**缺页频繁**导致 CPU 陷入内核态、**内存交换 (swap)** 导致 IO 瓶颈。这些问题往往交织出现，且其根因可能深藏于内核的页表管理、透明大页 (THP) 机制或 numa 内存分配策略中。eBPF 通过捕获内核内存事件，为这些问题提供了细粒度的诊断能力。

3.4.2 机制：page_fault 等内核事件的捕获与分析

SysTrace 在 L3 层通过 eBPF 挂载以下内核事件，构建内存行为的完整画像：

- **page_fault**：捕获用户态访问未映射页面时触发的缺页异常，用于计算**缺页率** (Page Fault Rate)。
- **mm_fault**：更细粒度的内存故障事件，包含故障地址与处理时延。
- **swap_in / swap_out**：捕获内存换入换出事件，识别内存压力导致的磁盘 IO。
- **kmem_alloc / kmem_free**：跟踪内核内存分配与释放，辅助诊断内存泄漏。

⁷ 视频画面时间区间：00:07:25–00:07:30。若为裁剪图，时间区间仍对应原视频画面。

基于这些事件，SysTrace 计算以下关键指标：

缺页率：在观测窗口 T 内，缺页异常次数 N_{pf} 与内存访问总次数 N_{access} 的比值：

$$P_{fault} = \frac{N_{pf}}{N_{access}} \times 100\%$$

其中， N_{access} 可通过硬件性能计数器（PMC）或 eBPF 对内存访问指令的采样估算。高缺页率通常意味着内存访问模式不连续或物理内存不足。

内存交换强度：单位时间内的 swap 页面数：

$$S_{rate} = \frac{N_{swap_in} + N_{swap_out}}{T}$$

若 S_{rate} 持续大于零，说明系统已启用 swap，训练性能将受到磁盘 IO 的严重拖累。

3.4.3 案例：透明大页（THP）导致的性能劣化

在某大模型训练任务中，SysTrace 检测到 step 时延在保存 checkpoint 阶段出现显著抖动。通过 eBPF 内存事件分析发现，该阶段内核的 `page_fault` 与 `mm_fault` 事件频率骤增，且处理时延远高于基线。进一步排查确认，系统开启了**透明大页（Transparent Huge Pages, THP）**特性：在内存压力较大的 checkpoint 阶段，THP 的内存回收与压缩机制频繁触发，导致大量页面合并与分裂操作，严重干扰了 AI 进程的正常执行。

该案例的启示在于：**内核内存优化特性并非总是对 AI 训练友好**。THP 在通用工作负载下可减少 TLB miss，但在大内存 footprint 的训练任务中，其后台压缩/回收开销可能超过收益。SysTrace 通过 eBPF 精准捕捉了这一隐蔽的根因。



图 8: eBPF 在 AI 性能观测中的主要观测点⁸

⁸视频画面时间区间：00:07:35–00:07:40。若为裁剪图，时间区间仍对应原视频画面。

eBPF 三大观测维度对比

观测维度	核心事件/探针	关键指标	典型根因
CPU 调度	<code>sched_switch</code> tracepoint	调度延迟、运行队列长度	非 AI 线程抢占 CPU
锁争用	<code>uprobe</code> （锁函数入口/出口）	锁等待时间、阻塞线程数	GIL 瓶颈、内存锁竞争
内存性能	<code>page_fault</code> , <code>swap_in/out</code>	缺页率、交换强度	THP 压缩、OOM、内存泄漏

3.5 本章小结

本节深入剖析了 SysTrace 的两大关键能力及其 eBPF 观测机制：

- 1. **常态化监控**以低开销持续采集 L0 层框架指标，约每 30 分钟检测一次 step 时延是否劣化，作为动态分层采集的“触发器”。
- 2. **动态分层采集**在检测到劣化后逐层开启 L1-L3 采集，先定界慢卡，再深挖通信、内存与 OS 层根因，实现了观测深度与性能开销的自适应平衡。
- 3. **eBPF CPU 调度分析**通过 `sched_switch` tracepoint 追踪 AI 线程的 CPU 占用与调度延迟，识别非核心线程（如 dataloader）的资源竞争。
- 4. **eBPF 锁争用分析**利用 `uprobe` 动态探测同步原语，量化锁等待时间与阻塞线程数，诊断 PyTorch 等框架中的 GIL 或自定义锁瓶颈。
- 5. **eBPF 内存性能分析**捕获 `page_fault` 等内核事件，分析缺页率与内存交换活动，定位透明大页（THP）等内核特性导致的性能劣化。

这三大 eBPF 观测维度并非孤立运作，而是与 L0-L2 的框架层、通信层、CANN 层数据协同，构成了一条从“现象发现”到“根因定位”的完整因果链。在下一节中，我们将通过三个实际的故障注入案例（NPU 算子占用、内存压力、CPU 调度干扰），展示 SysTrace 如何将上述能力转化为可操作的诊断结论。

4 效果展示：NPU、内存与 CPU 调度采集

前三章分别从挑战背景、技术架构与关键能力三个维度，系统阐述了 SysTrace 的设计思想与实现机制。本章将回归实践，通过三个精心设计的故障注入实验，展示 SysTrace 在真实场景中如何完成“慢节点定界”与“根因定位”。这三个案例分别对应 NPU 算子占用、内存压力与 CPU 调度干扰——它们恰好覆盖了 AI 训练中最常见、也最难排查的三类性能劣化来源。

本

章核心问题：当训练任务的 **step 时延** 出现异常增长时，SysTrace 如何利用 L0-L3 分层数据，在分钟级时间内定位到具体故障卡、故障类型乃至故障进程？

4.1 NPU 算子占用故障注入与定位

4.1.1 实验动机与配置

在大模型分布式训练中，NPU（或 GPU）是最核心的计算资源。任何导致单卡计算效率下降的因素——无论是硬件故障、驱动异常，还是其他任务抢占 AI Core——都会以“慢卡”形式拖累整个训练集群。由于分布式并行（尤其是张量并行 TP 与流水线并行 PP）要求组内卡严格同步，一张慢卡意味着同组所有卡必须空等，造成严重的资源浪费。

本实验选择 Llama2-3B 模型作为负载，采用 **DP1-TP4-PP2** 的并行配置：

- DP（Data Parallelism，数据并行）= 1：当前仅有一个数据并行组；
- TP（Tensor Parallelism，张量并行）= 4：每张 PP stage 内部有 4 张卡做张量切分；
- PP（Pipeline Parallelism，流水线并行）= 2：模型被切分为 2 个流水线阶段。

在此配置下，8 张 NPU 卡被组织为 2 个 PP stage，每个 stage 包含 4 张 TP 卡。TP 组内的 4 张卡需要频繁进行 AllReduce/AllGather 通信，因此任何一张卡的延迟都会直接阻塞其余 3 张。

4.1.2 故障注入方式

实验通过在第 3 号卡（编号 3）上启动一个小模型占用的方式，模拟 AI Core 被其他任务抢占的场景。这种注入方式无需修改训练代码，也无需重启主训练任务，仅通过在目标卡上额外启动一个轻量级推理进程，即可人为制造算力竞争。由于 NPU 的 AI Core 资源在同一时刻通常只能被一个进程独占使用，小模型的持续运行会导致训练进程在该卡上的算子执行时间显著拉长。

4.1.3 step 时延变化观测

故障注入前后，训练任务的 step 时延对比如下：

表 2: NPU 算子占用故障注入前后 step 时延对比

状态	step 时延 (ms)	增长率
Baseline（正常）	2501.4	—
故障注入后	3054.4	+22.1%

从数据可见，仅在三卡注入轻量级占用，整体 step 时延便从 2501.4ms 增长至 3054.4ms，增幅达 22.1%。这一结果直观地说明了“慢卡”对分布式训练的巨大拖累效应。

采集数据展示---NPU采集

注入AICore占用

● 模型配置DP1-TP4-PP2

注入故障方式：在3卡上启动小模型占用，导致主任务训练性能下降

```

117/ 50 | consumed samples: 100 | elapsed time per iteration (ms): 2501.4 | learning rate:
118/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
119/ 50 | consumed samples: 150 | elapsed time per iteration (ms): 2501.4 | learning rate:
120/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
121/ 50 | consumed samples: 200 | elapsed time per iteration (ms): 2499.0 | learning rate:
122/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
123/ 50 | consumed samples: 250 | elapsed time per iteration (ms): 2502.3 | learning rate:
124/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
125/ 50 | consumed samples: 300 | elapsed time per iteration (ms): 2782.1 | learning rate:
126/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
127/ 50 | consumed samples: 350 | elapsed time per iteration (ms): 3054.4 | learning rate:
128/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
129/ 50 | consumed samples: 400 | elapsed time per iteration (ms): 3057.0 | learning rate:
130/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
131/ 50 | consumed samples: 450 | elapsed time per iteration (ms): 3054.9 | learning rate:
132/ 50 | number of skipped iterations: 0 | number of nan iterations: 0
133/ 50 | consumed samples: 500 | elapsed time per iteration (ms): 3055.4 | learning rate:
134/ 50 | consumed samples: 550 | elapsed time per iteration (ms): 3053.6 | learning rate:

```

Step时延 baseline (ms)	Step时延 故障时刻均值 (ms)	Step时延 增长率
2501.4	3054.4	22.1%

1. 正常训练时，每个step平均耗时2501.4ms；发生性能劣化时，step耗时增加至3054.4ms，**同比增长22.1%**
2. 以模型训练需200000个step为例：

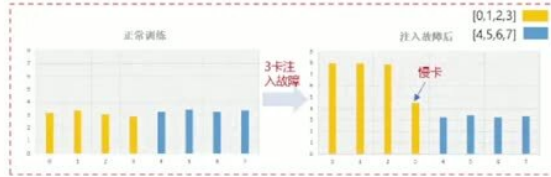
● 正常训练集群需耗时5.78天；

慢节点定义

- 方案：八卡分按照PP粒度分为两组，[0,1,2,3]和[4,5,6,7]



- TP组 hccIAllGather对比检测结果：



- 结论：

✓ 计算机会导致3卡的计算时间明显增加，同一组内的其他卡等待，导致通信慢

图 9: NPU 算子占用故障注入：step 时延对比⁹

4.1.4 慢卡定义：L0 + L1 数据协同分析

step 时延增长只是“症状”，真正的诊断任务是找到“病根”——哪张卡是慢卡？SysTrace 的解决思路是：利用 L0 框架层数据（关键阶段耗时）与 L1 通信层数据（通信算子耗时）进行交叉比对。

具体而言，在 TP4 组内，4 张卡的通信算子（如 AllReduce）理论上应几乎同时完成。若某张卡的 L1 通信耗时显著高于同组其他卡，则表明该卡的上游计算阶段存在延迟，导致通信发起时间被推迟。进一步结合 L0 层的 forward/backward 阶段耗时，即可确认该卡的计算阶段本身是否变慢。

在本实验中，SysTrace 通过 L0+L1 数据定位到**第 3 号卡（编号 3）为慢卡**：同组内 0-3 号卡中，其余卡的通信事件均在等待第 3 号卡到达同步点。第三张卡自身的计算时间正常（小模型占用并未导致其自身训练逻辑出错），但由于 AI Core 被抢占，其算子排队与执行时间被显著拉长，进而拖慢了整个 TP 组的进度。

⁹视频画面时间区间：00:09:30-00:09:35。若为裁剪图，时间区间仍对应原视频画面。

与 NPU 算子占用不同，内存问题的 symptoms 往往更加“隐蔽”：它不会导致某张卡的计算时间均匀拉长，而是表现为间歇性的卡顿——当内存压力触发内核的页回收或回写（writeback）时，整个训练线程会被阻塞，直到内存操作完成。

4.2.2 故障注入方式：NUMA 绑定

本实验采用 **numa 绑定**的方式注入内存压力。NUMA（Non-Uniform Memory Access）架构下，每个 CPU socket 拥有本地内存节点，跨节点访问会带来额外延迟。通过将训练进程绑定到特定 NUMA 节点，并在该节点上额外施加内存压力（例如启动一个大量分配内存的进程），可以人为制造本地内存带宽竞争与页错误风暴。

4.2.3 step 时延变化观测

内存压力注入前后，step 时延的变化如下：

表 3: 内存压力故障注入前后 step 时延对比

状态	step 时延 (ms)	增长率
Baseline（正常）	901.58	—
故障注入后	3026.52	+235.7%

step 时延从 901.58ms 暴增至 3026.52ms，增长率高达 235.7%。这一增幅远超 NPU 算子占用案例（22.1%），说明内存问题对训练性能的影响具有“非线性”特征：一旦触发内核的内存回收或 swap，阻塞时间可能长达数百毫秒甚至数秒。



图 11: 内存压力故障注入：step 时延对比¹¹

¹¹ 视频画面时间区间：00:10:45-00:10:50。若为裁剪图，时间区间仍对应原视频画面。

4.2.4 mm_fault

SysTrace 在 L3 OS 层采集了内存相关的内核事件，其中 **mm_fault**（内存管理缺页异常）时延是诊断内存压力的关键指标。实验对比了正常状态与故障注入后各张卡的 mm_fault 时延分布，发现以下规律：

1. **异常卡的 mm_fault 时延显著增长**：正常状态下，mm_fault 时延集中在微秒级；注入压力后，异常卡的 mm_fault 时延出现大量毫秒级甚至十毫秒级的离群点。
2. **统计数据分布更分散**：正常卡的 mm_fault 时延标准差较小，分布集中；异常卡的标准差显著增大，直方图呈现“拖尾”现象。
3. **信息熵作为判据**：为了量化这种“分散程度”，SysTrace 引入**信息熵**作为判据。信息熵越高，说明该卡的内存事件时间分布越不规则，越可能存在内存压力。

为

什么选用信息熵？

传统的均值/方差统计只能描述“集中趋势”与“离散程度”，但无法捕捉分布的“不规则性”。内存压力导致的 mm_fault 时延往往呈现多峰、拖尾、突发等复杂形态，信息熵恰好能量化这种“不确定性”。在实际部署中，SysTrace 会预先建立各卡的**信息熵基线 (baseline)**，当某卡的信息熵显著偏离基线时，即触发内存层级的细粒度采集。

信息熵的定义如下：

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

其中：

- X 为某张卡在一段时间窗口内的 mm_fault 时延随机变量；
- x_i 为将时延划分为 n 个离散区间后的第 i 个区间；
- $p(x_i)$ 为时延落在第 i 个区间的频率（概率估计）。

采集数据展示---内存采集

图 12: 内存定界: 信息熵对比¹²

4.2.5 根因判定: 内存回写错误

通过信息熵判据锁定异常卡后, SysTrace 进一步结合 L3 层的页表事件与 IO 事件, 确认该卡的内存压力导致了大量的内存回写 (**writeback**) 操作。正常情况下, 脏页 (dirty page) 的回写由内核后台线程异步完成, 对训练任务影响有限; 但在内存压力注入后, 可用内存急剧减少, 内核被迫频繁触发同步回写或直接回收 (direct reclaim), 导致训练线程被阻塞在内存管理路径上。

本案例的完整诊断链路为:

- 常态化监控发现 step 时延异常增长 (+235.7%);
- 触发 L3 内存事件采集, 获取各卡 mm_fault 时延分布;
- 计算信息熵, 发现某卡 (异常卡) 的信息熵显著高于同组其他卡;
- 下沉分析该卡的 page_fault、writeback、swap 等事件, 确认内存回写错误为根因;
- 结合 numa 绑定信息, 定位到具体 NUMA 节点的内存竞争。

4.3 CPU 调度干扰故障注入与定位

4.3.1 实验动机

在 AI 训练任务中, CPU 的角色常被低估: 它负责数据加载、预处理、通信协调、参数更新等关键环节。若某张卡绑定的 CPU 核心被非 AI 类线程 (如系统守护进程、日志采集进程、甚至其他用户的任务) 抢占, 则 AI 线程的调度延迟会增加, 进而影响整个训练流水线的效率。

¹²视频画面时间区间: 00:10:45-00:10:50。若为裁剪图, 时间区间仍对应原视频画面。

与 NPU 和内存问题不同，CPU 调度干扰的诊断难点在于“干扰源”通常是**外部进程**，而非训练任务自身。传统性能工具（如 Perf）可以采样 CPU 占用率，但难以在时序上精确关联“哪个进程在何时抢占了 AI 线程的 CPU”。

4.3.2 故障注入方式

本实验通过**资源竞争**的方式，对某张卡绑定的特定 CPU 核心注入高负载干扰。具体做法是：在该 CPU 上启动一个计算密集型进程（例如一个无限循环的浮点运算任务），使其持续占用 CPU 时间片，从而与 AI 训练线程形成调度竞争。

4.3.3 CPU 调度事件采集与分析

SysTrace 利用 eBPF 挂载 Linux 内核的 `sched_switch` tracepoint，实时采集 CPU 调度切换事件。每一次线程切换，eBPF 程序都会记录：

- 切换发生的时间戳；
- 被切换出的线程（prev_task）及其进程名；
- 被切换入的线程（next_task）及其进程名；
- 切换原因（如时间片耗尽、被抢占、阻塞等待等）。

通过对这些事件进行聚合分析，SysTrace 可以绘制出“AI 线程 vs 其他线程”的 CPU 占用时间线，精确识别哪些非 AI 进程在训练关键阶段抢占了 CPU 资源。

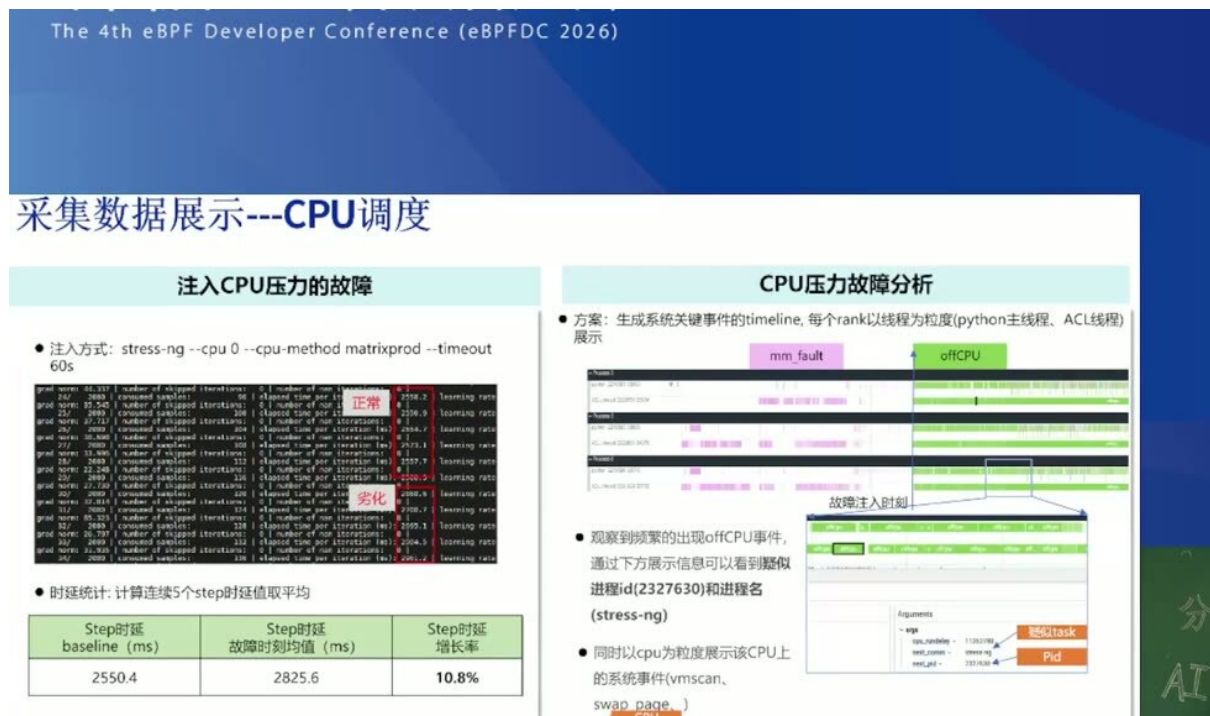


图 13: CPU 调度干扰识别：影响 AI 进程的进程名¹³

¹³视频画面时间区间：00:11:20-00:11:25。若为裁剪图，时间区间仍对应原视频画面。

sched_switch 的观测价值

sched_switch 是 Linux 调度器中最核心的 tracepoint 之一。与基于采样的 Perf 不同，eBPF 对 sched_switch 的追踪是**事件驱动**的：每一次调度切换都被精确记录，不会遗漏短时间的线程抢占。这使得 SysTrace 能够发现那些“一闪而过”但高频发生的干扰事件——例如，一个系统守护进程每 10ms 抢占一次 AI 线程，虽然单次抢占仅几百微秒，但累积效应足以导致 step 时延的显著劣化。

4.3.4 根因定位：识别具体进程名

在本实验中，SysTrace 通过 CPU 调度事件采集，成功识别出影响 AI 进程的具体进程名。分析结果显示：

1. 在注入干扰的 CPU 核心上，AI 训练线程的调度延迟（从就绪到实际运行的时间间隔）显著增加；
2. 高调度延迟的时段与某外部计算密集型进程的运行时段高度重合；
3. 该进程的进程名、PID、以及占用 CPU 的时间片数量均被 eBPF 精确记录。

这一结果的价值在于：它不仅告诉运维人员“CPU 有竞争”，而是直接给出“竞争源头是谁”。在传统运维中，这类问题往往需要反复试错（例如逐个排查系统进程、调整 cgroup 优先级）；而 SysTrace 通过 eBPF 的精确事件追踪，将诊断时间从小时级缩短至分钟级。

4.3.5 案例启示

CPU 调度案例展示了 SysTrace 在 L3 OS 层的一项独特能力：**跨进程影响溯源**。在共享集群环境中，同一台物理机上可能同时运行多个用户的任务，或存在大量系统后台进程。eBPF 的低开销特性使得 SysTrace 可以在生产环境中常态化采集调度事件，一旦发现某张卡的 AI 线程调度延迟异常，即可自动关联到具体的干扰进程，为后续的隔离、限流或迁移决策提供数据支撑。

4.4 本章小结

本章通过三个故障注入实验，系统展示了 SysTrace 在实际场景中的诊断能力：

表 4: 三类故障注入案例对比总结

故障类型	注入方式	核心观测指标	SysTrace 诊断手段
NPU 算子占用	小模型抢占 Core	AI step 时延 (+22.1%)	L0+L1 数据交叉比对, 通信等待定位慢卡
内存压力	NUMA 绑定注入内存竞争	step 时延 (+235.7%)	L3 mm_fault 时延分布 + 信息熵判据
CPU 调度干扰	CPU 高负载进程竞争	调度延迟增加	L3 sched_switch 事件追踪, 识别干扰进程名

三个案例共同印证了一个核心设计思想：**动态分层采集**。在故障未发生时，SysTrace 仅以低开销的 L0 层监控巡检；一旦检测到 step 时延劣化，便按需逐层开启 L1、L2、L3 的细粒度采集，最

终精准定位到具体故障卡、故障类型乃至故障进程。这种“由粗到细、按需深入”的策略，既保证了常态化监控的低开销，又确保了故障诊断的高精度。

生

产环境部署建议

故障注入实验的结果表明，内存压力对训练性能的影响最为剧烈（step 时延可增长 200% 以上），且 symptoms 具有高度非线性。因此，在生产环境中，建议对内存层（L3）的信息熵基线进行持续校准，并设置较敏感的告警阈值。同时，CPU 调度干扰虽然单次影响较小，但具有高频、随机、难以复现的特点，建议结合 `sched_switch` 事件的统计分布（如 P99 调度延迟）进行趋势监控，而非仅依赖瞬时阈值告警。

5 未来展望：业务拓扑画像与 Agent 调用链观测

在前几章中，我们系统梳理了 SysTrace 在大模型训练集群中的核心能力：从全栈协同分析架构到动态分层采集，再到 eBPF 在 CPU 调度、锁争用与内存性能三个维度的深度观测。然而，AI 基础设施的演进从未止步。随着大模型从训练阶段走向大规模推理部署，以及 AI Agent（智能体）应用的爆发式增长，性能观测正面临全新的挑战——系统内部的调用关系愈发复杂，传统观测手段逐渐力不从心。本章将聚焦 SysTrace 的两个前沿探索方向：一是面向 LLM 推理部署的**业务拓扑画像**，二是面向 AI Agent 的**调用链观测**。二者的共同目标，是从“盲人摸象”式的局部推测，迈向“全景导航”式的全局洞察。

5.1 LLM 推理部署的观测挑战

从训练到推理：观测场景的范式转移

大模型训练与推理在系统特征上存在显著差异。训练任务通常以数周甚至数月为周期，进程拓扑相对固定，观测重点在于识别慢节点、慢卡以及通信同步瓶颈。而推理部署则呈现出截然不同的面貌：

- **异构硬件混合**：推理场景下，CPU、NPU/GPU、DSA（Domain-Specific Accelerator）等多种计算单元往往协同工作，各自承担预处理、模型计算、后处理等不同阶段。硬件异构性导致性能瓶颈可能出现在任意一层，传统单一维度的 profiling 难以覆盖全貌。
- **Agent 多层嵌套**：现代推理服务不再是简单的“请求进、结果出”的单体架构。以 AI Agent 为例，一个用户请求可能触发 LLM 的多次迭代生成，每一次生成都可能调用外部工具（如搜索引擎、数据库、代码解释器），工具执行结果又作为上下文回传给 LLM 进行下一轮推理。这种**多层嵌套**的调用结构使得进程间关系动态变化，远超训练场景的静态拓扑。

传统端口战的局限

传统的性能观测往往依赖“端口战”思路——即通过监控固定端口流量、进程级指标或应用层日志来推断系统状态。然而，在异构硬件混合与 Agent 多层嵌套的场景下，这种方法面临三重困境：

1. **难以穿透内核**：应用层指标无法反映内核态的调度、IO、内存交换等关键事件；
2. **动态拓扑不可见**：进程与线程的创建、退出、通信关系瞬息万变，静态配置无法捕捉；
3. **系统状态成黑盒**：开发者对整体系统的资源交互缺乏宏观概念，性能调优与故障定位如同“盲人摸象”。

AI 学习任务的多进程协同困境

讲者特别指出，AI 学习任务（包括训练与推理）往往由**大量进程或线程协同完成**，其中既包含 AI 核心计算线程，也包含非 AI 类的辅助线程（如数据预处理、日志写入、监控上报等）。这些辅助进程若发生资源竞争或异常阻塞，可能间接拖慢整个 AI 任务的执行效率。

关键洞察

当性能劣化或故障发生时，开发者若缺乏对系统整体拓扑的宏观认知，往往难以判断问题的真实来源。一个看似无关的辅助进程，可能成为拖慢推理时延的“隐形杀手”。

5.2 基于 eBPF 的动态拓扑构建

面对上述挑战，SysTrace 提出的核心解决思路是：**基于 eBPF 的动态拓扑构建**。其本质是利用 eBPF 的无侵入探针能力，在内核态实时捕获系统关键事件，并据此动态构建出业务进程间的全量关系图谱。

eBPF 探针的无侵入数据采集

eBPF 程序可以安全地挂载到内核的关键路径上，Hook 以下类型的系统事件：

- **进程/线程生命周期**：通过追踪 `clone`、`fork`、`execve`、`exit` 等系统调用，捕获所有进程与线程的创建与退出事件；
- **网络通信行为**：通过追踪 `connect`、`accept`、`send`、`recv` 等网络相关系统调用，记录进程间的通信关系与流量特征；
- **文件 IO 交互**：通过追踪 `open`、`read`、`write`、`close` 等文件系统调用，刻画进程与存储资源的交互模式。

为什么 eBPF 适合动态拓扑构建？

传统动态拓扑构建通常依赖应用层埋点或代理（Agent）上报，存在以下问题：

- 需要修改应用代码或引入额外依赖，侵入性强；
- 只能观测已埋点的进程，容易遗漏辅助进程或第三方组件；
- 数据上报存在延迟，难以做到真正的实时。

而 eBPF 运行在内核态，对所有用户态进程一视同仁，无需修改应用代码即可实现**全量、实时、无侵入**的数据采集，天然适合构建动态业务拓扑。

动态构建业务拓扑关系图谱

基于上述采集到的原始事件，SysTrace 在用户态进行关联分析，构建出多维度的业务拓扑关系图谱：

1. **进程-线程层级树**：以推理服务主进程为根节点，逐层展开其子进程与线程，标注每个节点的生命周期起止时间；
2. **通信关系图**：以节点间的网络连接为边，标注连接方向、端口、协议类型及流量大小；
3. **资源交互图**：以文件 IO 和设备访问为边，标注进程与配置文件、模型权重、日志文件等资源的交互频次与时延。

特性一：刻画业务拓扑画像

在AI应用，特别是大语言模型（LLM）推理服务的部署与运维中，我们正面临前所未有的复杂性挑战。异构硬件混合、Agent多层嵌套等问题，使得传统观测手段难以穿透系统内核，导致内部状态成为难以窥探的“黑盒”。

背景必要性：从“盲人摸象”到“全景导航”的迫切需求

- **系统复杂性高**：在当今复杂的AI系统中，业务逻辑日益庞大，一个典型的AI任务往往涉及数十甚至上百个协同工作的进程与线程。
- **开发者视角局限**：开发者通常只能监控自己负责的模块或单个进程，无法看到整个系统的全貌，难以理解进程间的依赖关系和资源争抢情况。
- **问题定位困难**：当性能出现瓶颈时，很难判断是自身模块的问题，还是被其他进程“干扰”所致。这种“看不清、看不全、看不透”的现状，使得性能调优工作如同“盲人摸象”。

eBPF核心实现思路：基于eBPF的动态拓扑构建

- **eBPF探针无侵入数据采集**：在内核层面部署eBPF探针，Hook住进程/线程创建与退出、网络连接、文件IO等关键系统调用，实时捕获所有进程和线程的生命周期事件、通信行为和资源交互。
- **动态构建业务拓扑关系图谱**：将采集到的原始事件流进行实时处理和关联分析，动态构建一个包含进程、线程、容器、节点等多维度的拓扑关系图谱。
- **可视化呈现与智能分析**：将构建好的拓扑图通过可视化界面进行展示，并结合AI算法进行分析，自动识别关键路径、异常依赖和潜在的性能瓶颈。

图 14: 业务拓扑画像：背景与 eBPF 实现思路¹⁴

可视化呈现与智能分析

动态拓扑图谱的最终价值在于**可视化呈现与智能分析**。开发者可以通过交互式拓扑图，直观地看到：

¹⁴ 视频画面时间区间：00:11:50–00:11:55。若为裁剪图，时间区间仍对应原视频画面。

- 哪些进程是”热点节点”（连接数最多、IO 最频繁）；
- 哪些通信链路存在异常（时延突增、连接断开、重传率高）；
- 哪些辅助进程意外存活或频繁重启，可能暗示资源泄漏或配置错误。

更进一步，结合前序章节介绍的动态分层采集策略，当拓扑图中某一节点被标记为异常时，Sys-Trace 可以自动对该节点及其关联节点开启 L0-L3 的细粒度采集，实现从”宏观拓扑”到”微观根因”的逐层下钻。

核心价值：从”盲人摸象”到”全景导航”

动态拓扑构建的核心价值在于赋予开发者**过程洞察能力**——清晰呈现任务全链路中每一个进程、每一次通信、每一份资源交互；同时提升**可解释性**——当故障发生时，能够快速定位问题节点，而非在茫茫日志中大海捞针。

5.3 AI Agent 调用链观测

如果说业务拓扑画像解决的是”系统里有什么、它们在如何交互”的问题，那么 AI Agent 调用链观测则更进一步，试图回答”Agent 在做什么、为什么这么做”的问题。

打破 Agent 黑盒困境

当前业界对 AI Agent 的观测普遍停留在宏观指标层面，例如：

- 单位时间内的请求次数（QPS）
- 单次请求的总时延（Latency）
- 错误率与吞吐量

这些指标虽然能够反映 Agent 服务的整体健康度，但**无法揭示 Agent 内部的复杂工具调用逻辑**。一个 Agent 请求可能在内部经历了”LLM 生成 → 调用搜索工具 → 解析结果 → 再次调用 LLM → 调用代码解释器”等多个步骤，其中任意一步的时延抖动或失败都会导致最终用户体验劣化。缺乏对内部调用链的观测，Agent 系统本质上仍是一个**黑盒**。

特性二：观测Agent的调用链

当前对AI Agent的观测普遍停留在请求次数、总时延等宏观结果指标上，无法揭示其内部复杂的工具调用逻辑，导致Agent行为如同黑盒。我们需要超越浅层指标，通过观测完整的调用链，实现对Agent执行过程的深度洞察。

背景必要性：打破Agent的黑盒困境

- 逻辑复杂性：任务涉及多次工具调用，但中间的调用详情、耗时、IO均不可见。
- 观测局限性：宏观指标与内部操作割裂，无法定位任务延迟是源于LLM还是工具调用。

核心价值：迈向“Harness Engineering”

- 过程洞察：清晰呈现任务全链路，为调试优化提供深度依据。
- 可解释性：失败时快速定位问题节点，增强系统的鲁棒性与可信度。

eBPF 核心实现思路：构建“意图-行为”因果链

步骤一：系统捕获

部署eBPF探针，追踪Agent主进程及其所有子进程的系统调用（如execve, connect），捕获Agent的系统级行为。

步骤二：因果关联

设计关联引擎，将LLM的输出（意图）与后续的系统调用（行为）进行匹配，建立意图与行为的因果关联。

步骤三：链路可视化

将关联起来的“意图-行为”序列整合成一个完整的分布式追踪Span，并输出到标准的可观测性后端进行可视化分析。

图 15: Agent 调用链观测：背景与 eBPF 实现思路¹⁵

Harness Engineering 与 Agent 可观测性的兴起

讲者提到，近期 **Harness Engineering**（亦可理解为 Agent 工程化或智能体 harness 构建）在业界备受瞩目。随着 Agent 应用从原型走向生产环境，其可靠性、可解释性与可观测性成为关键议题。构建针对 Agent 的观测能力，不仅有助于**故障定位**，更能支撑**强化学习微调 (RLHF/SFT)** 与**后续输出优化**——通过分析 Agent 在真实环境中的行为轨迹，反哺模型训练，形成“观测-分析-优化”的闭环。

三步实现：系统捕获 → 因果关联 → 链路可视化

SysTrace 提出的 Agent 调用链观测方案，核心在于利用 eBPF 建立**意图-行为因果链**，具体分为三步：

第一步：系统捕获 通过 eBPF 探针捕获 Agent 主进程及其派生子进程的关键系统调用，重点追踪两类事件：

- `execve` / `execveat`：捕获 Agent 调用外部工具（如 Python 解释器、curl、数据库客户端）的行为；
- `connect` / `accept`：捕获 Agent 与外部服务（如搜索引擎 API、向量数据库）建立网络连接的行为。

```
1 // 追踪 execve 系统调用，捕获工具调用行为
2 SEC("tracepoint/syscalls/sys_enter_execve")
3 int trace_execve(struct trace_event_raw_sys_enter *ctx)
4 {
5     u32 pid = bpf_get_current_pid_tgid() >> 32;
```

¹⁵ 视频画面时间区间：00:13:20–00:13:25。若为裁剪图，时间区间仍对应原视频画面。

```

6     char comm[16];
7     bpf_get_current_comm(&comm, sizeof(comm));
8     // 记录进程名、PID、执行参数
9     // 后续用户态程序关联到对应 Agent 请求
10    return 0;
11}
12
13// 追踪 connect 系统调用, 捕获网络连接行为
14SEC("tracepoint/syscalls/sys_enter_connect")
15int trace_connect(struct trace_event_raw_sys_enter *ctx)
16{
17    u32 pid = bpf_get_current_pid_tgid() >> 32;
18    struct sockaddr_in *addr = (struct sockaddr_in *)ctx->args[1];
19    // 记录目标 IP、端口、协议类型
20    return 0;
21}

```

Listing 5: eBPF 探针示例：追踪进程执行与网络连接

第二步：因果关联 系统捕获得到的是离散的内核事件，下一步需要将这些事件与 Agent 的高层语义关联起来。核心思路是：

1. 在 Agent 框架侧（如 LangChain、AutoGPT 等），记录每一次 LLM 输出的**意图**（例如：“需要搜索最新天气”、“需要执行 Python 代码计算结果”）；
2. 在 eBPF 采集侧，记录该意图发出后一段时间内，Agent 进程执行的 `execve` 或 `connect` 事件；
3. 通过**时间窗口匹配与进程父子关系追踪**，将 LLM 输出（意图）与后续系统调用（行为）建立一一对应的**因果关系**。

意图-行为因果链的匹配原理

设 LLM 输出意图的时刻为 t_0 ，对应的 Agent 主进程为 P_{agent} 。eBPF 采集到的系统调用事件 e_i 包含时间戳 t_i 与进程 ID p_i 。因果关联的判定条件为：

- 进程归属： $p_i \in \text{Descendants}(P_{\text{agent}})$ ，即事件发生在 Agent 主进程或其子进程中；
- 时间邻近： $|t_i - t_0| < T_{\text{window}}$ ，其中 T_{window} 为预设的匹配窗口（通常数百毫秒至数秒）；
- 语义匹配：系统调用的目标（如搜索 URL、代码内容）与意图文本存在关键词或语义关联。

满足上述条件的事件序列，即构成一条完整的**意图-行为因果链**。

第三步：链路可视化 将因果关联后的调用链数据，以**分布式追踪 Span** 的形式进行结构化存储与可视化呈现。每个 Span 代表 Agent 执行流程中的一个逻辑步骤，包含以下字段：

- `trace_id`：唯一标识一次完整的 Agent 请求；
- `span_id`：当前步骤的唯一标识；

- `parent_id`: 父步骤的标识，用于构建调用层级；
- `operation`: 操作类型（如”LLM 生成”、”工具调用-搜索”、”工具调用-代码执行”）；
- `timestamp / duration`: 开始时间与耗时；
- `status`: 执行状态（成功/失败/超时）。

特性二：观测Agent的调用链

当前对AI Agent的观测普遍停留在请求次数、总时延等宏观结果指标上，无法揭示其内部复杂的工具调用逻辑，导致Agent行为如同黑盒。我们需要超越浅层指标，通过观测完整的调用链，实现对Agent执行过程的深度洞察。



图 16: Agent 调用链三步实现¹⁶

通过分布式追踪的可视化界面，开发者可以直观地看到一次 Agent 请求的完整执行路径：LLM 何时生成了什么意图、何时调用了什么工具、工具执行耗时多少、是否发生失败重试。这种**过程洞察与可解释性**，正是打破 Agent 黑盒困境的关键。

从”盲人摸象”到”全景导航”

无论是业务拓扑画像还是 Agent 调用链观测，SysTrace 的未来探索始终围绕一个核心命题：**让 AI 系统的内部状态从不可见变为可见，从不可解释变为可解释。**eBPF 作为内核态的”超级显微镜”，为这一命题提供了坚实的技术底座。

5.4 本章小结

本章展望了 SysTrace 在 LLM 推理部署与 AI Agent 观测两个前沿方向的探索：

1. **LLM 推理部署的观测挑战**：异构硬件混合与 Agent 多层嵌套使传统观测手段难以穿透系统内核，系统状态成为黑盒。AI 学习任务的多进程协同特性进一步加剧了故障定位的难度。
2. **基于 eBPF 的动态拓扑构建**：通过在内核态 Hook 进程/线程生命周期、网络连接与文件 IO 等关键系统调用，eBPF 实现了全量、实时、无侵入的数据采集。在此基础上动态构建业务拓扑关系图谱，赋予开发者从”盲人摸象”到”全景导航”的过程洞察能力。

¹⁶视频画面时间区间：00:13:20-00:13:25。若为裁剪图，时间区间仍对应原视频画面。

3. **AI Agent 调用链观测**: 当前 Agent 观测停留在宏观指标层面, 无法揭示内部复杂工具调用逻辑。SysTrace 提出**意图-行为因果链**的构建思路, 通过 eBPF 捕获系统调用 (`execve/connect`), 将 LLM 输出与后续系统调用匹配, 最终以分布式追踪 Span 的形式实现链路可视化, 打破 Agent 黑盒困境。

这两个方向的技术底座是 eBPF 的无侵入、全量、实时采集能力, 而共同的价值追求则是**过程洞察与可解释性**——让 AI 系统的每一次计算、每一次通信、每一次工具调用都清晰可见, 为性能优化、故障定位与模型迭代提供坚实的数据支撑。

6 总结与延伸

本章作为全篇的收尾, 旨在从三个维度对 SysTrace 的设计思想与技术价值进行系统性总结: 首先, 回顾演讲主体中提出的核心架构与关键机制; 其次, 提炼 Q&A 环节中关于产品形态、数据分析与采集策略的关键补充信息; 最后, 结合当前 eBPF 与 AI 系统交叉领域的前沿动态, 展望从“数据可视化”走向“智能根因分析”的演进路径, 并提出若干开放问题供读者进一步思考。

6.1 演讲核心要点回顾

SysTrace 的设计并非对现有工具的简单替代, 而是对 AI 训练集群性能观测范式的一次结构性重构。理解其技术价值, 需要把握以下四条贯穿全文的主线。

四条核心主线

- 1. **全栈协同分析架构**: 打破 L0 (框架层) 到 L3 (OS 层) 的数据孤岛, 实现跨层因果推理。
- 2. **动态分层采集**: 以“低开销常态化监控”为触发器, 按需逐层深入, 平衡观测精度与运行时开销。
- 3. **eBPF 三大观测维度**: CPU 调度、锁争用、内存性能, 覆盖系统层核心瓶颈。
- 4. **实际故障注入验证**: 通过 NPU 算子占用、内存压力、CPU 调度干扰三类案例, 验证端到端定位能力。

主线一: 从逐层孤立到全栈协同。传统性能诊断中, Python profiler、Perf、Ftrace、XPU Profiler 各管一段, 数据格式不统一、时间戳难对齐, 导致工程师需要在多个工具之间反复切换。SysTrace 通过统一的数据汇聚层, 将 L0 的 `dataloader/forward/backward` 阶段信息、L1 的集合通信算子 (`AllReduce/AllGather`) 时延、L2 的 CANN 层 HBM 申请释放轨迹, 以及 L3 的 OS 层调度/IO/内存/锁事件, 整合到同一套时间轴与语义框架下。这种“全栈协同”不是数据的简单堆叠, 而是建立了从上层行为到下层根因的因果推导链路。

主线二: 动态分层采集的“哨兵-深入”机制。若对所有层级、所有节点进行全量、全时的细粒度采集, 观测开销将不可接受。SysTrace 采用“哨兵”策略: 后台以约 30 分钟为周期巡检 `step` 时延等关键指标, 仅在检测到性能劣化后, 才触发逐层深入的采集流程——先定界到具体慢卡, 再开启 L1 通信层数据, 结合 L0 框架层信息判断是否为通信瓶颈, 最终下沉至 L2/L3 定位非 AI 类线程干扰或内存异常。这种设计将常态开销压至极低, 仅在“需要时”才支付深入观测的成本。

主线三: eBPF 在系统层的三大观测支柱。内核态的 eBPF 探针是 SysTrace 穿透 OS 黑盒的关键技术抓手:

- **CPU 调度**：通过 `sched_switch` tracepoint 追踪 AI 线程的 CPU 占用、调度延迟与运行队列长度，识别非核心线程的资源抢占。
- **锁争用**：利用 `uprobe` 动态探测同步原语（如 PyTorch 内存管理中的锁），量化锁等待时间与阻塞线程数，诊断 GIL 或自定义锁导致的同步瓶颈。
- **内存性能**：捕获 `page_fault` 等内核事件，分析缺页率与内存交换活动，定位内存泄漏、访问效率低下及 OOM 风险。

主线四：故障注入的实证闭环。技术架构的有效性最终需要通过实验验证。第 4 章中三个案例——Llama2-3B 模型上的 NPU 算子占用（`step` 时延增长 22.1%）、`numa` 绑定导致的内存压力（增长 235.7%）、CPU 高负载干扰——分别展示了 SysTrace 在慢卡定界、内存异常判据（信息熵）、非 AI 进程识别方面的端到端能力。

关键概念速查

- **动态分层采集**：先巡检、后深入，按需开启 L0-L3 采集，平衡精度与开销。
- **step 时延**：单次训练迭代（`forward+backward+ 优化器更新`）的耗时，是性能劣化检测的核心指标。
- **信息熵**：在内存故障定位中用作判据，异常卡的内存事件分布更分散，熵值更高。

6.2 Q&A 关键信息提炼

演讲结束后的 Q&A 环节进一步澄清了 SysTrace 的产品形态、数据分析现状与采集策略的工程化落地方式。这些信息对于理解 SysTrace 从“原型系统”到“生产工具”的演进路径至关重要。

6.2.1 使用形式：插件化定位与命令行控制

听众首先关心的是：SysTrace 以何种形态交付给用户？是侵入式地修改模型代码，还是独立运行的外部工具？

讲者明确回答：SysTrace 的核心定位是**类似插件的集成方案**。具体而言：

- **随训练任务启动**：SysTrace 在 AI 模型启动或启动之前即随之拉起，与训练进程生命周期绑定，无需用户手动干预启动流程。
- **命令行手动控制**：用户可通过命令行向服务端发送指令，指定需要采集的数据项、采集时长以及采集时延（延迟多久后开始采集）。这种设计既满足了“一键全采”的简便性，也支持“逐项精准采集”的灵活性。

这一定位的优势在于**非侵入性**：用户无需修改模型源码或训练脚本，SysTrace 作为旁路插件并行运行，降低了对现有训练流程的耦合度。

6.2.2 数据分析：从专家经验到大模型推荐

第二个关键问题聚焦于“采集之后怎么办”：SysTrace 是否具备自动分析能力，能否直接给出“这是慢节点”的诊断结论？

讲者的回答揭示了当前阶段的现实与未来的规划：

- **现状：依赖人工专家经验。**当前 SysTrace 将多层级数据汇总后，主要提供可视化呈现（如卡图、时间线、火焰图）。对于 OS 层采集的调度、IO、内存、锁等数据，由于其维度高、噪声大，具体的故障定位（例如“集合通信时长变长是否等于慢节点”）仍需要具备内核与 AI 框架双重背景的工程师进行人工分析。
- **未来：引入大模型进行智能推荐。**讲者明确表示，后续计划引入大模型，基于历史故障案例与专家标注数据，实现智能推荐与自动根因分析。这一演进路径的本质是：将当前沉淀在专家头脑中的“诊断 heuristic”转化为可规模化复用的模型能力。

当前阶段的认知边界

SysTrace 目前解决的是“采得到、看得见”的问题，尚未完全解决“看得懂、自动诊”的问题。读者在评估类似系统时，应区分“数据采集能力”与“智能分析能力”两个成熟度维度，避免将可视化呈现等同于自动根因定位。

6.2.3 按需采集的自动化策略：程序触发与人工控制的结合

第三位听众的问题直击“按需采集”的实现细节：采集是由程序自动控制，还是完全依赖人为选择？

讲者的回答澄清了 SysTrace 的工程化集成思路：

- **自动触发：**SysTrace 内置性能劣化检测逻辑，当 `step` 时延等关键指标超过阈值时，可自动触发分层采集流程。
- **人为控制：**同时保留命令行手动触发通道，供工程师在怀疑存在潜在问题时主动介入。
- **控制节点工程化集成：**SysTrace 将采集决策原则以接口形式暴露，用户在工程化集成时可通过自身的**控制节点**（如集群调度器、运维平台）统一编排，无需逐节点人工干预。

这种“自动 + 手动”的双模策略，既保证了大规模集群下的无人值守能力，又保留了专家深度介入的灵活性。

Q&A 核心要点速记

- **插件化定位：**随训练启动，命令行控制采集项/时长/时延。
- **专家经验：**当前故障定位依赖人工，OS 层数据尤其需要内核与 AI 双重背景。
- **大模型推荐：**未来计划引入 LLM 实现智能推荐与自动根因分析。
- **按需采集：**程序自动触发（劣化检测）+ 人为手动控制，通过控制节点统一编排。

6.3 技术趋势与开放问题

SysTrace 所代表的“eBPF + AI 全栈性能观测”方向，正处于快速演进的交叉路口。本节结合演讲内容与领域前沿，对这一方向的技术趋势进行结构化提炼，并提出若干值得持续关注的开放问题。

6.3.1 趋势一：从“数据汇总可视化”到“智能根因分析”

当前 SysTrace 以及同类系统的主流能力停留在**数据汇聚与可视化**层面——将分散在多层的指标、事件、trace 整合到统一的仪表盘上。这一阶段的瓶颈在于：

- 数据维度爆炸，人类工程师难以在有限时间内遍历所有关联。
- 根因与症状之间常存在非线性、跨层级的耦合关系，传统规则引擎难以覆盖。
- 专家经验难以规模化复用，不同团队重复踩坑。

演进方向是将“可视化”升级为“可解释的智能诊断”。具体路径包括：

1. **知识图谱化**：将 eBPF 采集的系统事件（调度延迟、锁等待、缺页）与 AI 框架语义（AllReduce 耗时、dataloader 阻塞）构建为因果知识图谱，实现跨层关联的形式化表达。
2. **大模型辅助推理**：利用 LLM 的上下文理解与模式识别能力，对历史故障报告、内核日志、火焰图进行联合推理，生成“最可能的根因假设”并给出置信度排序。
3. **闭环反馈**：将诊断结论与实际修复效果回灌到模型中，形成“观测-诊断-验证-进化”的闭环。

6.3.2 趋势二：eBPF 在 AI Agent 观测中的前沿价值

第 5 章已初步探讨了 SysTrace 在 LLM 推理部署与 AI Agent 观测方向的探索。从更宏观的视角看，eBPF 在这一领域的价值正从“性能优化”扩展到“行为理解”：

- **Agent 黑盒穿透**：AI Agent 通过工具调用（execute、connect、文件读写）与外部环境交互，传统日志只能看到“输入-输出”，eBPF 可捕获每一次系统调用的完整上下文，实现 Agent 内部决策链的透明化。
- **意图-行为因果链**：将 LLM 输出的“意图”（如“查询数据库”）与后续触发的系统调用序列进行匹配，建立高层语义与底层行为的因果关联。这不仅服务于故障定位，也为 Agent 的安全审计与合规检查提供了新手段。
- **动态拓扑构建**：在多 Agent 协作或异构硬件混合部署场景中，eBPF 可实时捕获进程/线程生命周期、网络连接与资源交互，动态构建业务拓扑画像，实现从“盲人摸象”到“全景导航”的跨越。

6.3.3 趋势三：系统层与框架层工具的互补共生

Q&A 中讲者特别强调：SysTrace **重点关注系统层性能瓶颈**（调度、IO、内存、锁），与上层框架侧工具（如 PyTorch Profiler、DeepSpeed Profiler）形成互补，而非替代。这一“分层分工、协同定位”的生态系统格局，在未来将持续深化：

- 框架层工具擅长定位算子级、图级、通信模式级的瓶颈；
- 系统层工具（如 SysTrace）擅长定位内核级、资源竞争级、硬件交互级的瓶颈；
- 两者的数据融合（例如将 PyTorch 的 autograd 时间戳与 eBPF 的 sched_switch 事件对齐）是下一步的关键工程挑战。

6.3.4 开放问题

基于以上趋势，以下问题值得研究者和工程师持续关注：

- 1. **观测开销的量化边界**：动态分层采集虽降低了常态开销，但在极端规模（万卡集群）下，触发后的 L3 层 eBPF 采集是否仍会对训练性能产生可感知的影响？如何建立开销模型并给出可证明的上界？
- 2. **跨层时间对齐的精度**：L0 的 Python 层时间戳与 L3 的内核 `bpf_ktime_get_ns()` 时间戳存在时钟源差异与系统调用延迟，如何在微秒级精度上实现跨层事件的因果排序？
- 3. **大模型诊断的可解释性与幻觉**：当引入 LLM 进行根因推荐时，如何确保其推理过程可解释、可验证，并避免在关键生产环境中因模型幻觉导致误判？
- 4. **eBPF 探针的移植性与标准化**：不同厂商的 XPU (NPU/GPU/TPU) 驱动与运行时差异显著，eBPF 探针的采集逻辑如何在异构硬件间保持可移植？行业是否需要一套面向 AI 工作负载的 eBPF 观测标准？
- 5. **按需采集与隐私安全的平衡**：eBPF 的高权限内核访问能力在提供强大观测力的同时，也带来了数据隐私与安全风险。如何在多租户集群中实现细粒度的探针权限隔离与审计？

本章核心 takeaway

SysTrace 的价值不仅在于其当前实现的“全栈采集 + 动态分层 + eBPF 观测”能力，更在于其揭示了一条清晰的演进路径：**从插件化的数据采集工具，逐步进化为具备智能诊断能力的 AI 全栈性能观测平台**。在这一过程中，eBPF 作为穿透系统黑盒的“显微镜”，将与大模型作为理解复杂系统的“放大镜”深度融合，共同推动 AI 基础设施的可观测性进入下一个阶段。

6.4 本章小结

本章从三个维度对 SysTrace 进行了系统性总结：首先回顾了全栈协同分析架构、动态分层采集机制与 eBPF 三大观测维度的核心设计；其次提炼了 Q&A 中关于插件化定位、专家经验分析、大模型智能推荐与按需采集策略的关键补充信息；最后结合 eBPF 与 AI 交叉领域的前沿动态，展望了从**数据可视化**走向**智能根因分析**的演进路径，并提出了观测开销边界、跨层时间对齐精度、大模型诊断可解释性、eBPF 探针标准化以及隐私安全等五个开放问题。SysTrace 的价值不仅在于其当前的技术实现，更在于它揭示了一条清晰的演进道路：以 eBPF 为显微镜穿透系统黑盒，以大模型为放大镜理解复杂关联，共同推动 AI 基础设施可观测性的持续进化。



图 17: 演讲结束页¹⁷

¹⁷ 视频画面时间区间：00:14:50–00:14:55。若为裁剪图，时间区间仍对应原视频画面。